

LC 1094: Car Pooling

Authored by Arpit Raj, LNMIIT Jaipur, July 11, 2026

Problem Overview

Companies: Zepto, Flipkart, Lyft

Input: `trips[i] = [num_passengers, from_i, to_i]` and the capacity of the vehicle.

Output: `true / false` if it is possible to pick up and drop off all passengers for all the given trips without exceeding the vehicle's capacity.

Example

Consider trips: `[[2, 1, 5], [3, 3, 7]]` with `capacity = 4`.

- At location 1: 2 people get in (current passengers = 2)
- At location 3: 3 people get in (current passengers = 2 + 3 = 5)

Since the overlap region has 5 people and $5 > 4$, the output is `false`.

Pattern Identified: Difference Array & Prefix Sum (Line Sweep)

Whenever a problem involves range updates (from start to end) and people getting on or off, always consider this pattern!

Approach 1: Events + Sorting

We can treat pickups and drop-offs as separate events. For every trip:

- **Pickup:** `(location, +passengers)`
- **Drop:** `(location, -passengers)`

By saving these events in a pair vector and sorting them, we can simulate the journey. If at any point the current passenger count exceeds capacity, return false.

TC: $O(n \log n)$

SC: $O(n)$

```
class Solution {
public:
    bool carPooling(vector<vector<int>>& trips, int capacity) {

        vector<pair<int,int>> events;

        for(auto &trip : trips){

            // Drop first (negative value)
            events.push_back({trip[2], -trip[0]});

            // Pickup
            events.push_back({trip[1], trip[0]});
        }

        sort(events.begin(), events.end());

        int cur = 0;

        for(auto event : events){

            cur += event.second;

            if(cur > capacity){
                return false;
            }
        }

        return true;
    }
};
```

Approach 2: Optimal (Difference Array + Prefix Sum)

The number of passengers only changes when someone gets in or out, not in between. We can use an array to record the *difference* in passengers at each location.

Let changes be at locations: 0, +2, 0, +3, 0, -2, 0, -3.

Taking the prefix sum gives the actual number of passengers at any given point: 0, 2, 2, 5, 5, 3, 3, 0. (Actual number = previous passengers + current change).

TC: $O(n)$

SC: $O(\text{max_location}) \approx O(1)$

```
class Solution {
public:
    bool carPooling(vector<vector<int>>& trips, int capacity) {
        int n=trips.size();
        vector<int> diff(1001,0); //farthest location tak ka diff array

        for(auto &trip:trips){
            int num=trip[0];
            int from=trip[1];
            int to=trip[2];

            diff[from]+=num;
            diff[to]-=num;
        }
        int curr=0;
        for(int i=0;i<1001;i++){
            curr+=diff[i];
            if(curr>capacity) return false;
        }

        return true;
    }
};
```